

GUÍA OPENCLAW

Skills

Enseñale a tu agente a hacer cualquier cosa

Qué es un skill · SKILL.md desde cero · Frontmatter y gating · Crear tu primer skill ·
Instalación desde ClawHub · Casos reales de la comunidad · Seguridad

Módulo 9 · KeroClaw · www.openagent.lat

9

MÓDULO 9

Skills*Enseñale a tu agente a hacer cualquier cosa*

I — Qué es un skill

Todo lo que vimos hasta ahora — AGENTS.md, SOUL.md, TOOLS.md, MEMORY.md, HEARTBEAT.md — define cómo el agente es. Los skills definen qué puede hacer más allá de sus capacidades base.

✦ La definición más precisa

Un skill es un playbook — un manual de instrucciones empaquetado que le enseña al agente cómo usar una herramienta, llamar a una API, ejecutar un workflow, o seguir un proceso específico de forma consistente.

No es un plugin que ejecuta código solo. No es una integración técnica. Es una carpeta con un archivo Markdown que el agente lee cuando necesita hacer esa tarea.

La diferencia con simplemente pedírselo en el chat: el skill lo hace de forma predecible, repetible, y con las guardrails que vos definiste.

La distinción crítica: herramientas vs skills

OpenClaw separa dos conceptos que parecen lo mismo pero son distintos:

Concepto	Qué es	Ejemplo
Herramienta (tool)	Una capacidad técnica que el agente puede ejecutar: leer archivos, correr comandos de terminal, llamar la API de Gmail, abrir el browser.	La herramienta Bash permite ejecutar comandos. La herramienta Gmail permite leer emails.
Skill	Un manual que le dice al agente cómo y cuándo usar esas herramientas para un workflow específico.	El skill 'weekly-report' le dice cómo combinar Gmail + Calendar + Bash para armar el reporte semanal.

💡 La analogía exacta

Tener la herramienta Bash es como tener instalado Python en tu computadora.

Tener un skill es como tener el script que sabe qué comandos correr, en qué orden, con qué parámetros, y qué hacer si algo falla.

Sin el skill, el agente improvisa cada vez. Con el skill, sigue el mismo proceso confiable siempre.

Por qué OpenClaw tiene más de 5.000 skills en ClawHub

Porque el formato es extremadamente simple: una carpeta con un archivo Markdown. No hace falta saber programar para crear uno básico. No hay SDK, no hay compilación, no hay deployment. Eso democratizó la creación: cualquier persona que entiende su propio workflow puede empaquetarlo como skill.

La comunidad pasó de 0 a más de 5.700 skills en ClawHub en pocas semanas. Hay skills para todo: GA4, Jira, Todoist, Obsidian, Home Assistant, trading, medicina, gestión inmobiliaria, análisis legal. Si tu caso de uso no existe todavía, lo creás en 20 minutos.

II — Estructura y ubicación de los skills

La estructura de carpetas

Un skill es siempre una carpeta con nombre en minúsculas y guiones. Adentro, SKILL.md es el único archivo obligatorio. Todo lo demás es opcional pero recomendado para skills complejos:

```
# Estructura de un skill completo:
$ ~/.openclaw/workspace/skills/
mi-skill/
  SKILL.md           # OBLIGATORIO — el manual del agente
  scripts/           # Opcional — scripts Python, Bash, etc.
    procesar.py
    backup.sh
  references/         # Opcional — documentación de apoyo
    formatos.md
    api-reference.md
  install.sh          # Opcional — setup de dependencias
  config.json         # Opcional — esquema de configuración
```

Qué va en cada carpeta

scripts/: los scripts que el agente ejecuta (Python, Bash, Node). El agente los referencia desde SKILL.md usando {baseDir} para construir el path correcto.

references/: documentación que el agente puede leer cuando necesita más contexto. Listas de formatos soportados, endpoints de APIs, guías de troubleshooting.

install.sh: comandos para instalar dependencias del skill. OpenClaw NO lo ejecuta automáticamente — el usuario lo activa manualmente (medida de seguridad).

Los tres lugares donde pueden vivir

1	<code>workspace/skills/</code>	Tu agente específico — máxima prioridad
2	<code>~/.openclaw/skills/</code>	Compartido entre todos tus agentes
3	Bundled skills	Los que vienen con la instalación — menor prioridad

Cuando hay conflicto de nombres — dos skills que se llaman igual — gana el de mayor precedencia. Esto es poderoso: podés ' parchear ' un skill bundled sin modificarlo, simplemente creando uno con el mismo nombre en tu workspace.

Skills por agente vs skills compartidos

En setups con múltiples agentes, tenés control granular:

```
# Skill solo para el agente 'ventas':  
$ ~/.openclaw/workspace-ventas/skills/crm-report/SKILL.md  
  
# Skill compartido por TODOS los agentes:  
$ ~/.openclaw/skills/gmail/SKILL.md  
  
# Extra: carpeta compartida adicional via config:  
# skills.load.extraDirs: ['~/mi-skills-pack']
```

Extra dirs para packs compartidos

Si querés un paquete de skills que aplique a múltiples agentes pero no a todos, usá `skills.load.extraDirs` en `openclaw.json`.

Estos tienen la menor precedencia de todas — `workspace` y `~/.openclaw/skills` siempre los pisan si hay conflicto.

SKILL.md — Anatomía completa del archivo

III — SKILL.md: anatomía completa

SKILL.md tiene dos partes: el frontmatter YAML (entre los --- iniciales) que le dice a OpenClaw cómo cargar el skill, y el cuerpo Markdown que le dice al agente qué hacer cuando lo usa. Vamos parte por parte.

El mínimo absoluto

Contenido del archivo	Qué hace / Por qué importa
---	Inicio del frontmatter YAML
name: mi-skill	Identificador único. En minúsculas, con guiones. Usado para /mi-skill y en skills.entries
description: Hace X cuando el usuario necesita Y.	CRÍTICO: determina cuándo el agente activa este skill automáticamente
---	Fin del frontmatter
# Mi Skill	Cuerpo Markdown — instrucciones para el agente
## Qué hace	
Descripción del workflow.	

⚠ La descripción es el trigger de activación automática

El agente lee todas las descripciones de skills disponibles para decidir cuál usar. Una descripción vaga o genérica puede causar que el skill nunca se active, o que se active cuando no debe.

Regla: empezá con el verbo de acción y terminá con el contexto de uso. 'Genera reportes semanales de actividad combinando emails, calendario y logs de proyectos' activa mucho mejor que 'skill de reportes'.

Frontmatter completo: todos los campos

Contenido del archivo	Qué hace / Por qué importa

name: weekly-report	Nombre único del skill — minúsculas y guiones
description: Genera reporte semanal combinando emails, entradas de calendario y logs del proyecto.	Trigger de activación automática — sé específico
Usar cuando se pida resumen, reporte o semana.	El agente lee esto para decidir si usar el skill
	Incluí las palabras que el usuario diría
user-invocable: true	Expone /weekly-report como slash command (default: true)
disable-model-invocation: false	Si true: solo slash command, nunca auto-activación

```

metadata: {"openclaw": {
  "emoji": "🇺🇸",
  "requires": {
    "bins": ["python3"],
    "env": ["CALENDAR_API_KEY"],
    "config":
["channels.email.enabled"]
  }
}}
---
```

JSON en una sola línea — requerimiento del parser
 Ícono en la UI del dashboard
 Gates — si no se cumplen, el skill no se carga
 Binario que debe existir en PATH del sistema
 Variable de entorno requerida — no carga sin ella
 Ruta en openclaw.json que debe ser truthy

Cerrar el JSON en la misma línea — sin saltos

Gating: filtros de carga automática

Gating es el sistema que OpenClaw usa para decidir si un skill está disponible en un entorno dado. Si los gates no se cumplen, el skill simplemente no aparece — ni en el listado, ni en el prompt del agente.

Gate	Qué verifica	Caso de uso típico
bins	Binarios en PATH del sistema	skill-pdf solo carga si python3 está instalado. skill-video solo carga si ffmpeg está disponible.
anyBins	Al menos uno de los binarios listados	Skill que acepta jq o python3 para parsear JSON — alcanza con uno.
env	Variable de entorno existe (o está en config)	Skill de email solo carga si GMAIL_TOKEN está definida. Sin la key, no aparece.
config	Ruta en openclaw.json debe ser truthy	Skill de browser solo carga si channels.browser.enabled es true en la config.
os	Sistema operativo específico	Skill que usa comandos específicos de Linux — evita que se cargue en Windows/WSL sin soporte.
always: true	Ignorar todos los gates — siempre disponible	Skills de diagnóstico o utilidades que no dependen de nada externo.

💡 Por qué el gating es importante

Sin gating, un skill que requiere Python3 pero se usa en una máquina sin Python generaría errores confusos. Con gating, el skill simplemente no se carga y el agente ni lo intenta.

Esto también reduce el token overhead: skills que no aplican al entorno no se inyectan en el prompt. Cada skill que no está disponible = tokens que no se gastan.

El cuerpo: qué escribir para que el agente lo siga

El cuerpo del SKILL.md es donde definís el workflow. No hay formato obligatorio, pero hay un patrón que la comunidad refinó que produce resultados consistentes:

✗ Skill débil (vago)

✓ Skill robusto (concreto)

```
name: reporte
description: Hace reportes.
---
```

```
# Reporte
```

```
Cuando el usuario pide un reporte,
juntá información relevante y
generá un resumen.
```

```
name: weekly-report
description: Genera reporte semanal de
  actividad combinando emails recibidos,
  eventos del calendario y tareas cerradas.
  Usar cuando se pida resumen de la semana.
---
```

```
## Inputs requeridos
```

```
- Rango de fechas (default: lunes-viernes
  actual)
```

```
## Workflow
```

1. Leer emails recibidos en el rango
2. Leer eventos del calendario en el rango
3. Leer tareas cerradas en el rango
4. Agrupar por categoría: reuniones, decisiones, pendientes, logros
5. Generar resumen en formato Markdown con secciones diferenciadas

```
## Guardrails
```

- Máximo 400 palabras en el resumen
- Si no hay datos en alguna sección: omitirla
- No inventar información ausente

El skill robusto funciona mejor porque: los inputs son explícitos, el workflow tiene pasos numerados con comandos concretos, y hay stop conditions definidas para casos de error.

IV — Slash commands: activación manual

Cada skill con `user-invocable: true` (el default) genera automáticamente un slash command. Es la forma de activar un skill de forma explícita, sin depender de que el agente lo detecte automáticamente.

Cómo funciona

```
# Un skill llamado 'weekly-report' genera:
$ /weekly-report

# Con argumentos:
$ /weekly-report semana del 17 al 21 de febrero

# Activación automática sin slash:
# 'generame el reporte de esta semana' → el agente detecta el skill

# Listar todos los skills disponibles:
$ openclaw skills

# Forzar skill desde terminal para testing:
$ openclaw agent --message 'usá mi skill weekly-report'
```

Command dispatch: bypass del modelo

Hay un modo avanzado donde el slash command NO pasa por el modelo — va directo a una herramienta. Útil para comandos utilitarios donde no necesitás razonamiento:

Contenido del archivo	Qué hace / Por qué importa
<pre>--- name: disk-check description: Muestra uso de disco del sistema. command-dispatch: tool command-tool: exec command-arg-mode: raw ---</pre>	
	<i>Bypass del modelo — va directo a la herramienta</i>
	<i>Herramienta a invocar</i>
	<i>Pasa los argumentos del slash command tal cual</i>
<code>df -h</code>	<i>El cuerpo se convierte en el comando a ejecutar</i>

Cuándo usar command-dispatch

Para comandos de diagnóstico rápido: `/disk-check`, `/ping-server`, `/gateway-status`.

Para acciones deterministas donde el razonamiento del modelo no agrega valor — y donde querés cero latencia y cero tokens.

NO lo uses para workflows complejos donde el agente necesita decidir, adaptar, o interpretar output.

V — Instalar skills de ClawHub

ClawHub es el registro público de skills de OpenClaw — más de 5.700 skills creados por la comunidad, organizados por categorías. Para instalar un skill de ClawHub, hay tres métodos:

Método 1: clawhub install (CLI)

```
# Instalar un skill por su slug:
$ clawhub install gmail
$ clawhub install ga4
$ clawhub install github-prs

# Instalar en una carpeta específica:
$ clawhub install gmail --target ~/.openclaw/skills/

# Actualizar todos los skills instalados:
$ clawhub update --all

# Sincronizar (escanear y publicar actualizaciones):
$ clawhub sync --all
```

Método 2: desde el chat directamente

Una de las formas más potentes: pegás la URL del repositorio GitHub del skill en el chat y el agente lo instala solo:

```
# En el chat:
# 'Instalá este skill: https://github.com/openclaw/skills/tree/main/gmail'

# El agente:
# 1. Lee el SKILL.md del repositorio
# 2. Crea la carpeta en tu workspace
# 3. Descarga los archivos
# 4. Confirma la instalación
```

Método 3: manual (recomendado para skills propios o auditados)

```
# Crear la estructura manualmente:
$ mkdir -p ~/.openclaw/workspace/skills/mi-skill
$ nano ~/.openclaw/workspace/skills/mi-skill/SKILL.md

# OpenClaw detecta el cambio automáticamente
# (el skills watcher está activo por defecto)
# No necesitás reiniciar el Gateway.
```

⚠ **SEGURIDAD: leé el SKILL.md antes de instalar cualquier skill externo**

Los skills tienen acceso a las mismas herramientas que el agente — pueden ejecutar comandos de terminal, leer archivos, enviar emails. Un skill malicioso puede hacer daño real.

En 2026 se reportaron skills maliciosos en ClawHub que instalaban malware o eliminaban emails. ClawHub tiene integración con VirusTotal, pero el escaneo no es infalible.

Reglas mínimas: (1) lee el SKILL.md completo antes de instalar, (2) revisa qué herramientas declara en el frontmatter, (3) preferí skills con muchas estrellas en GitHub y reviews en ClawHub, (4) usá ClawDex para escanear skills antes de activarlos.

Cómo ver qué skills están cargados

```
# Listar todos los skills disponibles en el agente actual:
$ openclaw skills

# Ver detalle de un skill específico:
$ openclaw skills info gmail

# Ver por qué un skill NO está disponible (gating):
$ openclaw skills --debug
```

Token overhead de los skills

Cada skill disponible se inyecta como XML en el system prompt del agente. El costo es predecible y calculable:

```
# Fórmula de costo (en caracteres, ~4 chars = 1 token):
# total = 195 + Σ (97 + len(name) + len(description) + len(location))

# Ejemplo práctico:
# 10 skills con descriptions de ~80 chars promedio:
# = 195 + 10 × (97 + 10 + 80 + 30) = 195 + 2170 = ~2365 chars = ~591 tokens

# Para reducir el overhead:
# 1. Deshabilitar skills que no usás: enabled: false en openclaw.json
# 2. Usar allowBundled para whitelist de bundled skills
# 3. Usar gating estricto para que solo carguen cuando aplican
```

Crear tu primer skill desde cero — guía paso a paso

VI — Crear tu primer skill desde cero

Ejemplo concreto: un skill de reporte semanal que combina emails, calendario y tareas cerradas, y genera un resumen en Markdown. Es un caso universal — cualquier usuario puede adaptarlo a sus fuentes de datos.

Paso 1 — Crear la estructura

```
# Ir al workspace del agente:
$ cd ~/.openclaw/workspace/skills/

# Crear la carpeta del skill:
$ mkdir weekly-report
$ cd weekly-report

# Crear el archivo principal:
$ touch SKILL.md
```

Paso 2 — Escribir el SKILL.md

Contenido del archivo	Qué hace / Por qué importa

name: weekly-report	Slug único — coincide con el nombre de la carpeta
description: Genera reporte semanal de actividad	Trigger de activación — sé específico
combinando emails recibidos, eventos del calendario y tareas cerradas del período.	Podés continuar en líneas siguientes con sangría
Usar cuando se pida resumen o reporte semanal.	Incluí las palabras clave que el usuario diría
metadata: {"openclaw": {"emoji": "📅",	JSON en una sola línea
"requires": {"env": ["CALENDAR_TOKEN"]}}}	Gate: no carga sin el token de calendario

## Inputs requeridos	
- Rango de fechas (default: semana actual)	Con default explícito el agente no pregunta
## Workflow	
1. Leer emails recibidos en el rango	Fuente 1
2. Leer eventos del calendario en el rango	Fuente 2

```
3. Leer tareas cerradas en el rango
4. Agrupar por categoría:
   reuniones, decisiones,
   pendientes, logros
5. Generar resumen en Markdown con
   secciones
```

```
## Guardrails
```

```
- Máximo 400 palabras en el resumen
- Si no hay datos en una sección:
  omitirla
- No inventar información ausente
```

Fuente 3

Categorías predefinidas — evita inventar

Formato de output explícito

Límite de longitud — evita output gigante

Stop condition para secciones vacías

Anti-alucinación

Paso 3 — Proveer el token sin escribirlo en el archivo

```
# En openclaw.json — inyectar el token de forma segura:
{
  "skills": {
    "entries": {
      "weekly-report": {
        "enabled": true,
        "env": {
          "CALENDAR_TOKEN": "tu-token-aqui"
        }
      }
    }
  }
}

# Alternativa: variable de entorno del sistema (más segura)
$ export CALENDAR_TOKEN='tu-token-aqui'
```



Nunca pongas secretos en SKILL.md

SKILL.md se inyecta en el system prompt en cada conversación. Si ponés un token o API key ahí, el modelo la ve, los logs la ven, y si compartís el skill la ven todos.

La forma correcta: declarar el nombre de la variable en metadata.requires.env (para el gating) y proveer el valor en skills.entries.<n>.env o como variable de entorno del sistema.

Paso 4 — Verificar que el skill se cargó

```
# Listar skills disponibles:
$ openclaw skills

# Buscar el skill específico:
$ openclaw skills | grep weekly-report

# Si no aparece — ver por qué falló el gating:
$ openclaw skills --debug

# Testear desde el chat:
# 'generame el reporte de esta semana'
# o activar explícitamente con slash command:
```

```
# /weekly-report semana del 17 al 21 de febrero
```

Paso 5 — Iterar basándose en los logs

El skill watcher recarga SKILL.md automáticamente cada vez que lo guardás — no hace falta reiniciar el Gateway. Editá, guardá, probá en el chat. El ciclo de iteración es rápido.

```
# Ver logs del Gateway en tiempo real:  
$ tail -f /tmp/openclaw/openclaw-$(date +%Y-%m-%d).log  
  
# Buscar entradas del skill específico:  
$ grep 'weekly-report' /tmp/openclaw/openclaw-*.log
```

VII — Casos reales de la comunidad

Estos son casos documentados por usuarios reales de OpenClaw — no ejemplos teóricos. Cada uno muestra la paleta de lo que los skills permiten hacer.

Productividad personal

— El caso de uso más popular según encuestas

Skill que corre cada mañana a las 7am (vía cron) combinando: clima del día, primeros 3 eventos del calendario con notas de preparación, 3 titulares de noticias de las fuentes elegidas, top 3 tareas abiertas en Linear/Notion, y una línea de 'qué priorizar hoy' basada en el contexto.

Resultado: todo en menos de 200 palabras entregado por Telegram. Sin abrir ninguna app.

El skill requiere: Gmail + Google Calendar skills, weather API, RSS reader, y el task manager preferido del usuario.

— Usuario documentó el proceso en X

Primer día de uso: el agente limpió 10.000 emails acumulados — archivó newsletters, respondió auto-replies, y dejó solo los que requerían acción humana.

El skill define categorías, reglas de archivo, y un template de respuesta para cada tipo de email. El guardrail clave: 'nunca eliminar, siempre archivar. Nunca enviar sin aprobación.'

Continúa corriendo diariamente manteniendo el inbox limpio de forma autónoma.

Negocios y trabajo

— Solo founder, documentado en X

4 agentes especializados: 'Milo' (estrategia y coordinación), 'Josh' (desarrollo y arquitectura), uno de marketing (research y contenido), uno de negocio (métricas y pricing). Cada uno con su propio workspace y modelo optimizado.

Memoria compartida para decisiones de proyecto, memoria individual para contexto de cada rol. Trabajan en paralelo y se coordinan vía agent-send.

Todo controlado desde un solo chat de Telegram. Disponibles 24/7 en VPS.

— Desarrollador publicó en

ClawHub el mismo día

El usuario necesitaba consultar métricas de Google Analytics desde el chat. No existía el skill en ClawHub. Lo construyó en ~20 minutos, lo testeó, y lo publicó.

Ahora cualquier usuario puede ejecutar 'clawhub install ga4' y consultar sus analytics en lenguaje natural.

Este es el ciclo que la comunidad promueve: encontrás un gap, construís el skill, pagás para adelante.

Hogar y vida personal

documentó en showcase

— Usuario con wife and kids,

Setup completo: plan de comidas para 365 días del 2026, listas de compras ordenadas por tienda y pasillo, pronóstico del tiempo integrado para días de parrilla o sopa, catálogo de recetas por cocinero familiar.

El agente recuerda en el briefing matutino y nocturno si hay que planificar la cena o comprar ingredientes.

Estimación del usuario: ahorra más de 1 hora semanal en coordinación familiar.

Assistant + servidor propio

— Usuario con Home

Todo integrado en un solo chat de Telegram: emails, Home Assistant (luces, alarmas, temperatura), homelab vía SSH, lista de compras, Apple Notes.

Un skill para cada sistema. El agente entiende instrucciones en lenguaje natural y las traduce a las APIs correspondientes.

El usuario lo describió como 'la punta del iceberg' — comenzó con una integración y fue expandiendo.

Automatizaciones técnicas

del poder del formato

— Usuario lo documentó como ejemplo

El agente se conectó a Jira por su cuenta y construyó un skill desde cero porque no existía en ClawHub en ese momento.

El usuario solo le dijo 'conectate a Jira' y el agente: leyó la API docs, escribió el SKILL.md, lo instaló en el workspace, y confirmó que funcionaba.

Ejemplo concreto de que el skill format es tan simple que el propio agente puede escribir skills para sí mismo.

— caso documentado por múltiples medios

— Aj Stuyvenberg

El agente scrapeó inventarios de concesionarias locales, llenó formularios de contacto, y durante varios días hizo competir a los vendedores entre sí enviando PDFs de cotizaciones y

pidiendo a cada uno que mejorara la oferta del otro.

Resultado: \$4.200 por debajo del precio de lista del Hyundai Palisade 2026. El usuario solo apareció para firmar.

No es un skill de uso general, pero ilustra qué pasa cuando combinás skills de browser automation + email + memoria + autonomía prolongada.

VIII — Hoja de referencia rápida

Comandos de terminal más usados

Comando	Qué hace
<code>openclaw skills</code>	Lista todos los skills disponibles en el agente actual
<code>openclaw skills info <nombre></code>	Detalle de un skill: descripción, gates, estado
<code>openclaw skills --debug</code>	Muestra por qué skills no están cargados (gates fallidos)
<code>clawhub install <slug></code>	Instala un skill desde ClawHub al workspace
<code>clawhub update --all</code>	Actualiza todos los skills instalados desde ClawHub
<code>/nombre-del-skill</code>	Activa el skill manualmente desde el chat (slash command)
<code>openclaw agent --message 'usá X'</code>	Testea un skill desde terminal

Preguntas frecuentes

Pregunta	Respuesta
¿Qué es un skill?	Un playbook en Markdown que le enseña al agente a ejecutar un workflow específico de forma consistente.
¿El skill ejecuta código solo?	No. Le dice al agente qué herramientas usar y cómo. Las herramientas deben estar habilitadas por separado.
¿Tengo que reiniciar al crear un skill?	No. El skills watcher recarga automáticamente al guardar el SKILL.md.
¿Cómo activo un skill?	Automáticamente (el agente detecta el contexto) o manualmente con <code>/nombre-del-skill</code> .
¿Puedo tener el mismo skill en dos lugares?	Sí, pero gana el de mayor precedencia: <code>workspace > managed > bundled</code> .
¿Cuánto pesan los skills en tokens?	~195 + 97 chars por skill. Con 10 skills de descriptions normales: ~600 tokens aprox.
¿Cómo desactivo un bundled skill?	<code>skills.entries.<nombre>.enabled: false</code> en <code>openclaw.json</code> .
¿Puedo crear un skill sin saber programar?	Sí. Un skill mínimo es solo nombre + descripción + instrucciones en lenguaje natural.
¿Los scripts en <code>scripts/</code> se ejecutan automáticamente?	No. El agente los llama desde SKILL.md usando la herramienta Bash. Necesita Bash en tools.
¿Es seguro instalar skills de ClawHub?	No sin revisión. Lee siempre el SKILL.md antes de instalar y verificá en ClawDex/VirusTotal.

🚩 La guía core de OpenClaw está completa

Con este módulo cerramos el ciclo completo del workspace: AGENTS.md, SOUL.md, TOOLS.md, USER.md, IDENTITY.md, MEMORY.md, HEARTBEAT.md, y Skills.

Cada pieza encaja: el agente sabe cómo operar (AGENTS), quién es (SOUL + IDENTITY), quién sos vos (USER), qué tools puede usar (TOOLS), qué recuerda (MEMORY), cuándo actuar solo (HEARTBEAT), y cómo ejecutar workflows específicos (Skills).

El ecosistema de ClawHub crece decenas de skills por día. El paso siguiente es explorar, instalar los que resuelven tus casos de uso, y crear los que no existen todavía.